



D S P 4 0 4

Unidad I Arreglos y Colecciones

Universidad de El Salvador

S e m a n a 3

Arrays · List<T> Matrices

Unidimensionales

Multidimensionales

List<T>

Temas de hoy:

- ¿Qué es un arreglo y para qué sirve?
- Declarar, inicializar y acceder por índice
- Recorrer con for y foreach
- Operaciones: máximo, mínimo, suma, promedio
- Arreglos multidimensionales (matrices)
- List<T> — el arreglo dinámico

¿Qué lograrás al terminar esta clase?



Explicas qué es un arreglo y cuándo usarlo

La diferencia entre guardar 1 dato y guardar 100 del mismo tipo.

Declaras e inicializas arreglos en C#

Sintaxis correcta, acceso por índice, propiedad Length.

Recorres arreglos con for y foreach

Cuándo usar cada uno según la operación que necesites.

Calculás máximo, mínimo, suma y promedio

Operaciones clásicas sobre colecciones de datos.

Usas matrices y List<T>

Datos en tabla (2D) y colecciones que crecen dinámicamente.

Bloque 1

¿Qué es un Arreglo?

El problema que resuelve antes de ver el código

El problema: guardar muchos datos del mismo tipo

✗ Sin arreglos — el caos

```
double nota1 = 8.5;
double nota2 = 9.0;
double nota3 = 6.5;
double nota4 = 7.0;
double nota5 = 10.0;
// ... y así hasta nota30
// ¡30 variables distintas!
// Imposible de manejar 😱
```

CS

Problemas:

- 30 variables distintas que nombrar
- Imposible recorrerlas con un ciclo
- Agregar una nota = cambiar el código
- Calcular promedio = escribir 30 sumas

✓ Con arreglos — elegante

```
double[] notas = {
    8.5, 9.0, 6.5, 7.0, 10.0,
    // ... hasta 30 valores
};

// Recorrer TODOS con 3 líneas:
foreach (double n in notas)
    Console.WriteLine(n);

// Promedio en 2 líneas:
double suma = notas.Sum();
double prom = suma / notas.Length;
```

CS

Ventajas:

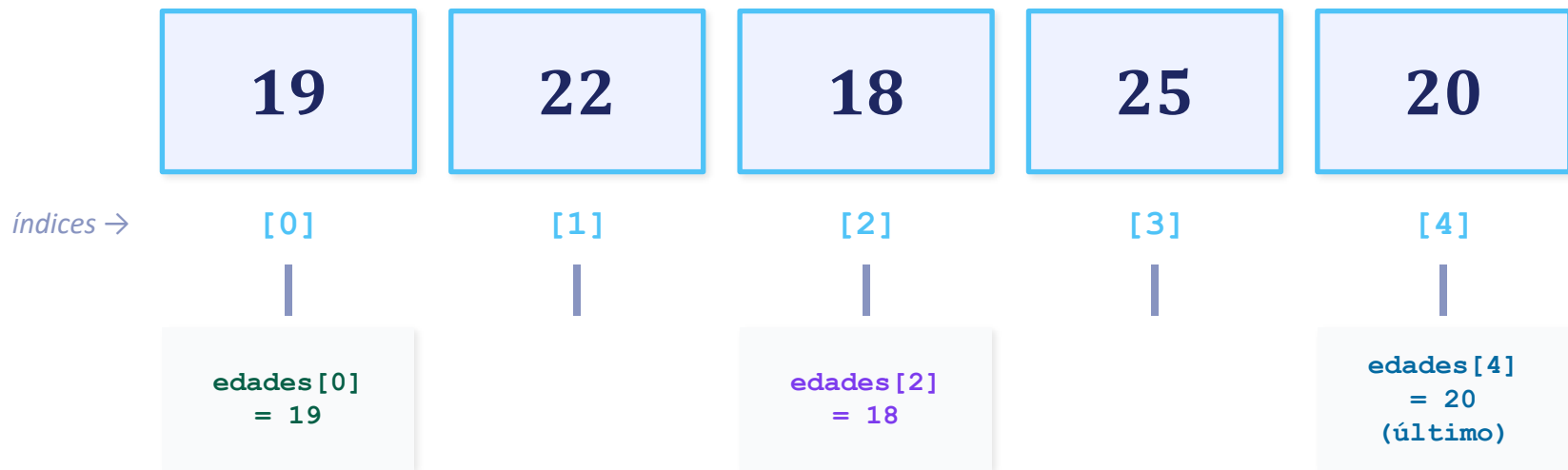
- Una sola variable para todos los datos
- Se recorre con for o foreach
- Length da el tamaño automáticamente
- Agregar datos es trivial

¿Qué es un arreglo? — anatomía visual



Un arreglo es una estructura que guarda VARIOS valores del MISMO tipo bajo UN solo nombre. Cada valor tiene una posición (índice) que empieza en 0.

```
int[] edades = { 19, 22, 18, 25, 20 };
```



Error clásico: si el arreglo tiene 5 elementos, el último índice es [4], no [5]. Acceder a [5] causa `IndexOutOfRangeException`.

Bloque 2

Arrays Unidimensionales

Declarar, inicializar, acceder y recorrer

Declarar e inicializar un arreglo — todas las formas

```
// — FORMA 1: declarar con tamaño (valores en 0/null por defecto) —
int[] numeros = new int[5];           // [0, 0, 0, 0, 0]
string[] nombres = new string[3]; // [null, null, null]

// — FORMA 2: declarar e inicializar con valores —
int[] notas = { 8, 9, 6, 10, 7 };      // más común ✓
double[] precios = new double[] { 1.5, 2.0, 3.75 };

// — FORMA 3: declarar con var —
var colores = new string[] { "rojo", "verde", "azul" };

// — ACCEDER Y MODIFICAR por índice —
Console.WriteLine(notas[0]);           // 8   (primero)
Console.WriteLine(notas[4]);           // 7   (último)
notas[2] = 8;                          // modificar el valor en [2]

// — PROPIEDAD Length —
Console.WriteLine(notas.Length); // 5
Console.WriteLine($"Último índice válido: {notas.Length - 1}"); // 4

// — PRIMER Y ÚLTIMO elemento —
Console.WriteLine(notas[0]);           // primero
Console.WriteLine(notas[notas.Length-1]); // último
```

CS



notas.Length - 1 siempre apunta al último elemento. Memoriza esta expresión.

Recorrer un arreglo — for vs foreach

Regla práctica: usa foreach cuando solo lees, usa for cuando necesitas el índice o modificas valores.

for — cuando necesitas el índice

CS

```
string[] alumnos = {"Ana","Beto","Carla"};

// Imprimir con número de posición:
for (int i = 0; i < alumnos.Length; i++)
    Console.WriteLine($"{i+1}. {alumnos[i]}");
// 1. Ana 2. Beto 3. Carla

// Modificar valores:
int[] notas = {5, 8, 4, 9, 6};
for (int i = 0; i < notas.Length; i++)
    if (notas[i] < 6) notas[i] = 6; // mínimo 6

// Recorrer al revés:
for (int i = notas.Length-1; i >= 0; i--)
    Console.Write(notas[i]+" ");
```

foreach — cuando solo lees

CS

```
int[] notas = {8, 6, 9, 5, 10, 7};

// Suma y promedio:
int suma = 0;
foreach (int nota in notas)
    suma += nota;

double promedio = (double)suma / notas.Length;
Console.WriteLine($"Prom: {promedio:F2}");

// Buscar el máximo:
int max = notas[0];
foreach (int n in notas)
    if (n > max) max = n;

Console.WriteLine($"Máximo: {max}");

// ❌ No puedes modificar dentro del foreach:
// foreach (int n in notas) n = 0; // Error
```


Operaciones clásicas sobre arreglos

CS

```
int[] ventas = { 450, 820, 310, 950, 680, 120, 750 };

// — SUMA —
int suma = 0;
foreach (int v in ventas) suma += v;
Console.WriteLine($"Suma total: {suma}");           // 4080

// — PROMEDIO —
double promedio = (double)suma / ventas.Length;
Console.WriteLine($"Promedio: {promedio:F2}");      // 582.86

// — MÁXIMO Y MÍNIMO —
int max = ventas[0], min = ventas[0];
foreach (int v in ventas) {
    if (v > max) max = v;
    if (v < min) min = v;
}
Console.WriteLine($"Mayor venta: {max}");           // 950
Console.WriteLine($"Menor venta: {min}");           // 120

// — BÚSQUEDA LINEAL —
int buscar = 680;
int posicion = -1;
for (int i = 0; i < ventas.Length; i++) {
    if (ventas[i] == buscar) { posicion = i; break; }
}

if (posicion >= 0)
    Console.WriteLine($"Venta {buscar} encontrada en posición {posicion}");
else
    Console.WriteLine("No encontrada");
```



Patrón Max/Min: inicializa max y min con el PRIMER elemento (ventas[0]), nunca con 0. Si todos son negativos, 0 daría un

Ejercicio explicado — Estadísticas de Notas de un Grupo

```
// ESTADÍSTICAS DE NOTAS — DSP404 Semana 3
const int TOTAL = 5;
double[] notas = new double[TOTAL];

// — Leer notas con validación —
Console.WriteLine($"Ingresa {TOTAL} notas (0.0 a 10.0):
");
for (int i = 0; i < TOTAL; i++) {
    double n;
    bool valida = false;
    while (!valida) {
        Console.Write($"  Nota {i+1}: ");
        if (double.TryParse(Console.ReadLine(), out n) && n >= 0 && n <= 10) {
            notas[i] = n;
            valida = true;
        } else
            Console.WriteLine("  🚫 Ingresa un número entre 0.0 y 10.0.");
    }
}

// — Calcular estadísticas —
double suma = 0, max = notas[0], min = notas[0];
int aprobados = 0;

foreach (double n in notas) {
    suma += n;
    if (n > max) max = n;
    if (n < min) min = n;
    if (n >= 6.0) aprobados++;
}
double promedio = suma / TOTAL;

// — Mostrar resultados —
Console.WriteLine("
===== ESTADÍSTICAS =====");
Console.WriteLine($"Promedio:      {promedio:F2}");
Console.WriteLine($"Nota mayor:  {max:F1}");
Console.WriteLine($"Nota menor:  {min:F1}");
Console.WriteLine($"Aprobados:   {aprobados}/{TOTAL}");
Console.WriteLine($"Reprobados:  {TOTAL - aprobados}/{TOTAL}");
```

Bloque 3

Arreglos Multidimensionales (Matrices)

Datos organizados en filas y columnas

¿Qué es una matriz? — el concepto



Una matriz es un arreglo con dos índices: [fila, columna]. Piénsala como una hoja de cálculo o una tabla de calificaciones.

Notas de 3 alumnos en 4 materias:

CS

	Mat	Español	Ciencias	Historia
Alumno 0	9	8	7	10
Alumno 1	6	9	8	← notas[1,2] = 8
Alumno 2	10	7	9	8



notas.GetLength(0) = número de filas. notas.GetLength(1) = número de columnas. (Diferente al .Length de arrays 1D.)

Recorrer matrices — for anidado

CS

```
int[,] notas = {
    { 9, 8, 7, 10 },
    { 6, 9, 8, 7 },
    { 10, 7, 9, 8 }
};

int filas = notas.GetLength(0); // 3 alumnos
int cols = notas.GetLength(1); // 4 materias

// — Imprimir toda la tabla —————
Console.WriteLine("Alumno\tMat\tEsp\tCie\tHis");
for (int f = 0; f < filas; f++) {
    Console.Write($"Alumno {f}\t");
    for (int c = 0; c < cols; c++)
        Console.Write($"{notas[f,c]}\t");
    Console.WriteLine();
}

// — Promedio por alumno (recorrer por filas) —————
for (int f = 0; f < filas; f++) {
    int suma = 0;

    for (int c = 0; c < cols; c++) suma += notas[f, c];
    double prom = (double)suma / cols;
    Console.WriteLine($"Alumno {f}: promedio = {prom:F1}");
}

// — Mayor nota de TODA la matriz —————
int max = notas[0, 0];
for (int f = 0; f < filas; f++)
    for (int c = 0; c < cols; c++)
        if (notas[f, c] > max) max = notas[f, c];
Console.WriteLine($"Nota más alta: {max}");
```

Bloque 4

List<T> El arreglo dinámico

Cuando no sabes cuántos elementos tendrás

¿Por qué existe List<T>? — Array vs List



Un array tiene tamaño FIJO: una vez creado con 5 elementos, no puedes agregar un sexto. List<T> es flexible: crece y encoge según necesites.

	Array int[]	List<T> List<int>
Tamaño	Fijo al crear	Dinámico (crece/encoge)
Agregar	✗ No se puede	✓ .Add(valor)
Quitar	✗ No se puede	✓ .Remove() / .RemoveAt()
Acceder	arr[0]	list[0] (igual)
Tamaño	arr.Length	list.Count
Usar cuando	Tamaño conocido	Tamaño desconocido

List<T> — sintaxis y métodos principales

CS

```
// — Crear una List —
var productos = new List<string>();           // lista vacía
var notas     = new List<double> { 8.5, 9.0, 6.0 }; // con valores

// — Agregar elementos —
productos.Add("Laptop");
productos.Add("Mouse");
productos.Add("Teclado");
Console.WriteLine(productos.Count);    // 3

// — Acceder por índice (igual que array) —
Console.WriteLine(productos[0]);        // "Laptop"
productos[1] = "Mouse Inalámbrico";    // modificar

// — Quitar elementos —
productos.Remove("Laptop");             // por valor
productos.RemoveAt(0);                  // por índice

// — Buscar —
bool existe = productos.Contains("Teclado"); // true
int pos     = productos.IndexOf("Teclado");  // 0

// — Recorrer con foreach —
foreach (string p in productos)
    Console.WriteLine($"{p}");

// — Ordenar —
notas.Sort();                          // ascendente
notas.Reverse();                       // descendente
Console.WriteLine($"Mejor nota: {notas[0]}");
```

Métodos más usados

.Add (v)	Agrega al final
.Insert (i, v)	Inserta en posición
.Remove (v)	Quita por valor
.RemoveAt (i)	Quita por índice
.Contains (v)	¿Existe el valor?
.IndexOf (v)	Posición del valor
.Sort ()	Ordena ascendente
.Clear ()	Vacía la lista
.Count	Número de elementos



¡Tu turno! — Matriz de Notas

30 minutos | Individual o en parejas
Lee el enunciado en el siguiente slide



EJERCICIO — Registro de Notas con Matriz



Parte 1 — Arreglo 1D de notas

- Declara un arreglo de 6 notas enteras con valores entre 1 y 10.
- Recórrelo con foreach y muestra cada nota.
- Calcula e imprime: suma, promedio, nota más alta, nota más baja.
- Cuenta cuántas notas son ≥ 6 (aprobadas) y cuántas no.

Parte 2 — Matriz de notas (3×3)

- Declara una matriz `int[3,3]` con 3 alumnos y 3 materias.
- Imprime la matriz completa en forma de tabla (con for anidado).
- Calcula el promedio de notas de cada alumno (recorre por filas).
- Encuentra la nota más alta de toda la matriz.

Parte 3 — Lista dinámica de productos ★

- Crea una `List<string>` vacía. Con un while, pide nombres de productos hasta que el usuario escriba 'fin'.
- Al terminar, muestra todos los productos con su número, cuántos hay en total, y si 'Laptop' está en la lista.

Errores comunes — Semana 3

Los que TODOS cometes al aprender arrays

❌ IndexOutOfRangeException — índice fuera de rango

```
int[] n = {1,2,3};  
Console.WriteLine(n[3]); // ❌ solo hay  
[0],[1],[2]
```

```
Console.WriteLine(n[n.Length-1]); // ✅ último  
elemento
```

🚫 *IndexOutOfRangeException: Index was outside the bounds of the array*

❌ Inicializar max/min con 0 en lugar de notas[0]

```
int max = 0; // ❌ ¿y si todas las notas son  
negativas?  
foreach(int n in arr) if(n>max) max=n;
```

```
int max = arr[0]; // ✅ empieza desde el  
primer elemento real  
foreach(int n in arr) if(n>max) max=n;
```

🚫 *Resultado incorrecto sin error visible – el peor tipo de bug*

❌ Usar .Length en matrices en vez de GetLength()

```
int[,] m = new int[3,4];  
for(int i=0; i<m.Length; i++) // ❌  
Length=12, no 3
```

```
for(int f=0; f<m.GetLength(0); f++) // ✅  
filas=3  
    for(int c=0; c<m.GetLength(1); c++) // ✅  
        cols=4
```

🚫 *Comportamiento inesperado – itera 12 veces en vez de por filas/columnas*

Para la próxima clase — Guía de ejercicios



Guía de ejercicios graduada — entrega para la próxima clase

E1

Fácil

Suma y promedio de arreglo

E2

Fácil

Mayor y menor de un arreglo

E3

Fácil

Buscar un valor en arreglo

E4

Medio

Invertir un arreglo

E5

Medio

Matriz de temperaturas

E6

Medio

Lista de compras dinámica

E7

Difícil

Tabla de calificaciones completa

E8

Difícil

Gestión de inventario con List